Lab Report: Synthesizing Birdsong with the RP2350

Contents

1 Introduction **2**

2 Design and Testing Methods **3**
2.1 Design Approach and Development . . . 3
2.2 Hardware Design . . . 4
2.2.1 RP2350, Potentiometer, and Keypad . . . 4
2.2.2 DAC and Audio Output . . . 4
2.3 Software Design . . . 5
2.3.1 DDS and Real-Time Audio . . . 6
2.3.2 Keypad Scanning and Debouncing . . . 8
2.3.3 Modes, Recording, and Playback . . . 9
2.4 Testing, Debugging, and AI Use . . . 10
2.4.1 Incremental Testing and Debugging . . . 10
2.4.2 AI-Assisted Development and Verification . . . 12

3 Documentation **13**
3.1 Hardware Reference . . . 13
3.2 Software Organization . . . 13

4 Results **13**
4.1 Functional and Signal Results . . . 13
4.2 Birdsong and Spectrogram Evaluation . . . 16
4.3 Merlin Evaluation and Limitations . . . 16

5 Conclusions **17**

6 Work Distribution **17**

A Complete Source Code **17**

1 Introduction

The purpose of this lab was to design a real-time birdsong synthesizer using the RP2350 microcontroller. Our system uses direct digital synthesis (DDS) to generate audio rather than playing prerecorded sound. A slide potentiometer allows the user to continuously control the generated frequency from 0–10 kHz, while a 4×3 matrix keypad provides controls for muting, recording, and replaying frequency trajectories. The user can record movements of the potentiometer onto numbered keypad keys and later replay those trajectories to reproduce the rapid frequency changes characteristic of birdsong. The RP2350 generates the audio waveform at a nominal 50 kHz synthesis rate and transmits samples through SPI to an MCP4822 digital-to-analog converter (DAC), which produces the analog audio output. The final design combines this real-time synthesis with keypad debouncing, multiple recording slots, playback, and sequence-based composition to provide an interactive method of constructing birdsong-like calls. We used the northern cardinal as the primary reference for evaluating the frequency patterns produced by the system.

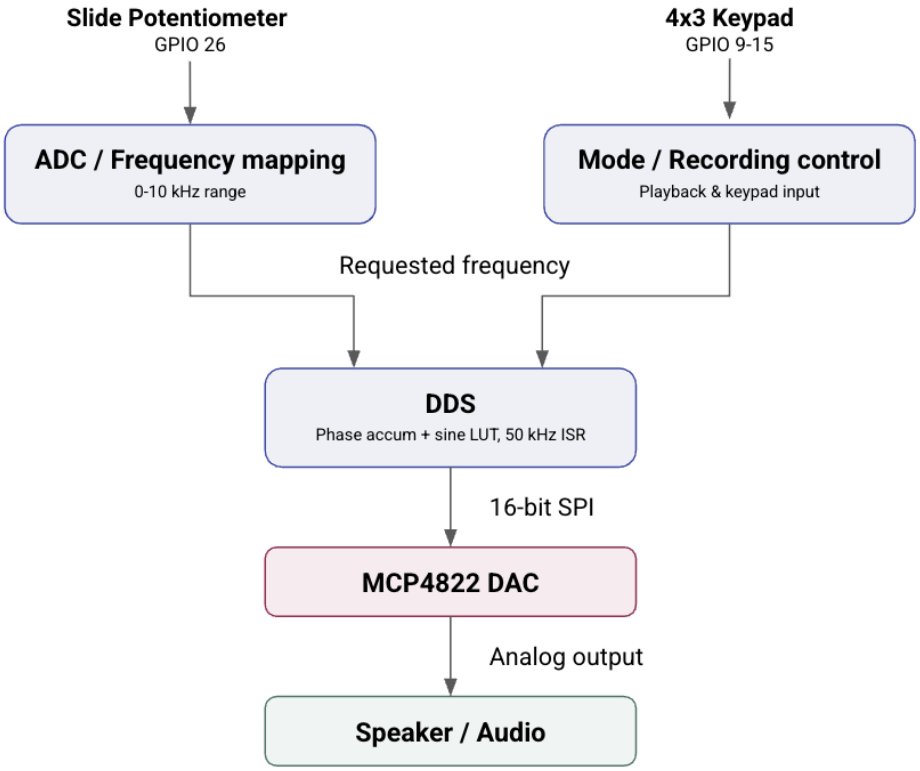


Figure 1: High-level system architecture.

2 Design and Testing Methods

2.1 Design Approach and Development

We developed the birdsong synthesizer incrementally over three weeks, with each checkpoint adding functionality to a working version of the previous system. During Week 1, we brought up the primary hardware and audio path on the RP2350. We configured SPI communication with the MCP4822 DAC and used a 50 kHz timer interrupt to generate a sine wave using direct digital synthesis (DDS). We then integrated the slide potentiometer through the RP2350 ADC and mapped its position to an output range of 0–10 kHz, allowing the generated tone to be controlled continuously by the user.

During Week 2, we expanded the system to support interactive recording and playback using a 4×3 matrix keypad. We implemented keypad scanning and debouncing, assigned the 0 key to mute and unmute the output, and used the $*$ key to arm recording for the next numbered key press. The numbered keys 1–9 were assigned independent recording slots. In Record Mode, holding a numbered key recorded the frequency trajectory produced by moving the potentiometer; in Tone Mode, the same key could replay the stored trajectory.

During Week 3, we extended these recorded trajectories toward complete birdsong sequences. We added Compose Mode to combine recordings into ordered sequences and increased their playback speed to reproduce the rapid frequency modulation present in bird calls. We then constructed cardinal-like frequency patterns and evaluated the final system using oscilloscope measurements, spectrograms, and an attempted comparison with the Merlin Bird ID application. Throughout all three weeks, hardware and software were tested incrementally so that each subsystem could be verified before being integrated into the final design.

2.2 Hardware Design

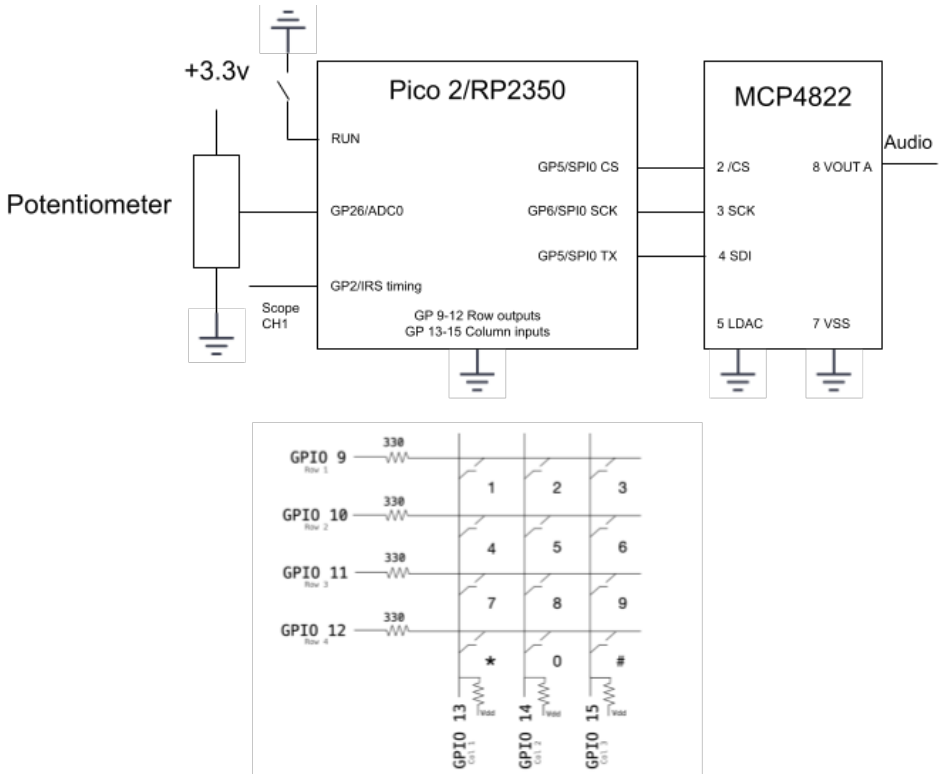


Figure 2: System wiring overview; GPIO assignments appear in Table 2.

2.2.1 RP2350, Potentiometer, and Keypad

The system is built around the RP2350 microcontroller (Raspberry Pi Pico 2), which serves as the central controller for the entire system. It provides the ADC, GPIO, hardware timer, and SPI peripherals used throughout the design.

The potentiometer is wired between +3.3V and ground, with its wiper connected to GPIO 26, configured as ADC channel 0. This wiper voltage provides an analog control signal proportional to the slider's physical position. The RP2350's ADC is 12-bit, so readings range from 0 to 4095 across the full 0 to 3.3V input range.

The keypad is a 4×3 matrix, wired as shown in Figure 2. The four row lines are connected to GPIO 9 through GPIO 12, each through a 330 Ω series resistor. The three column lines are connected to GPIO 13 through GPIO 15, configured as inputs with the RP2350's internal pull-up resistors enabled. This physical arrangement allows the twelve keys to be sensed using only seven GPIO pins rather than one pin per key; the scanning algorithm and debounce state machine that make use of this wiring are described in Section 2.3.

2.2.2 DAC and Audio Output

Synthesized audio samples are sent from the RP2350 to an MCP4822 dual-channel, 12-bit digital-to-analog converter over SPI0. Three GPIO pins are used for this communication:

GPIO 5 drives the DAC's chip select (CS), GPIO 6 provides the SPI clock (SCK), and GPIO 7 carries the serial data output (SDI/MOSI) to the DAC. SPI0 is configured for 16-bit transfers at 20 MHz, fast enough to complete a full sample transmission well within the required sample period.

Only DAC Channel A is used to output the synthesized audio waveform. Because the DDS sine table produces signed sample values, each sample is shifted upward by 2048 before transmission so that it falls within the DAC's unsigned 12-bit output range. The DAC's Channel A output (VOUT A) is connected to the tip terminal of a 3.5 mm stereo audio jack, with the sleeve terminal tied to circuit ground; the second audio channel is left unconnected.

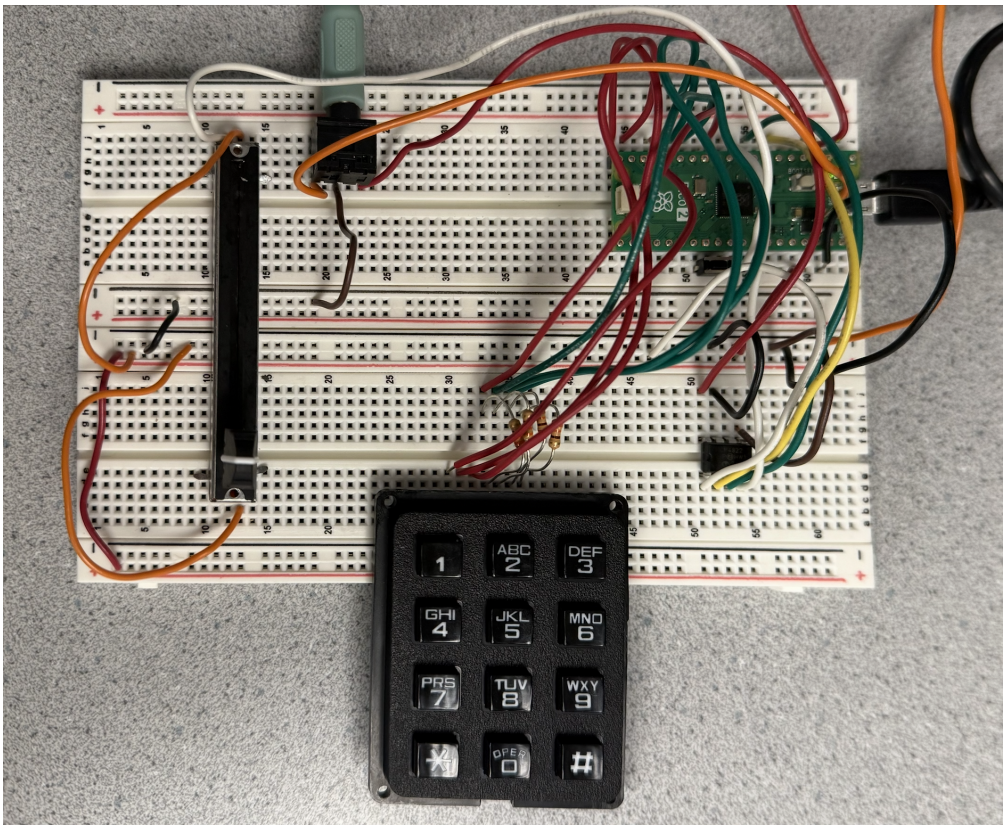


Figure 3: Completed hardware assembly.

2.3 Software Design

The software was organized around two lower-rate protothreads and a high-frequency interrupt service routine, as shown in Figure 4. This separation allowed user input, recording, and playback logic to run independently from the timing-critical audio synthesis.

The keypad thread runs at approximately 100 Hz and is responsible for scanning and debouncing the 4×3 keypad. Confirmed key events update the shared system state, including the current operating mode, mute state, and active recording or playback slot. The frequency thread uses a 10 ms yield during live control and normal recording (approximately 100 Hz), and a 1 ms yield during playback (approximately 1 kHz). These are nominal rates; execution and scheduling add to each yield interval. It reads and maps the potentiometer input, records

frequency samples into the nine recording buffers, retrieves stored samples during playback, and selects the frequency that should currently be generated based on the system state.

After selecting the output frequency, the frequency thread calls `set_frequency()`, which converts the requested frequency into the 32-bit phase increment stored in `phase_incr_main`. The 50 kHz DDS interrupt reads this shared value and performs the time-critical waveform generation, including phase accumulation, sine-table lookup, DAC command construction, and SPI transmission to the MCP4822. In this architecture, the lower-rate threads determine *what* frequency should be produced, while the ISR determines the individual samples required to generate that frequency in real time.

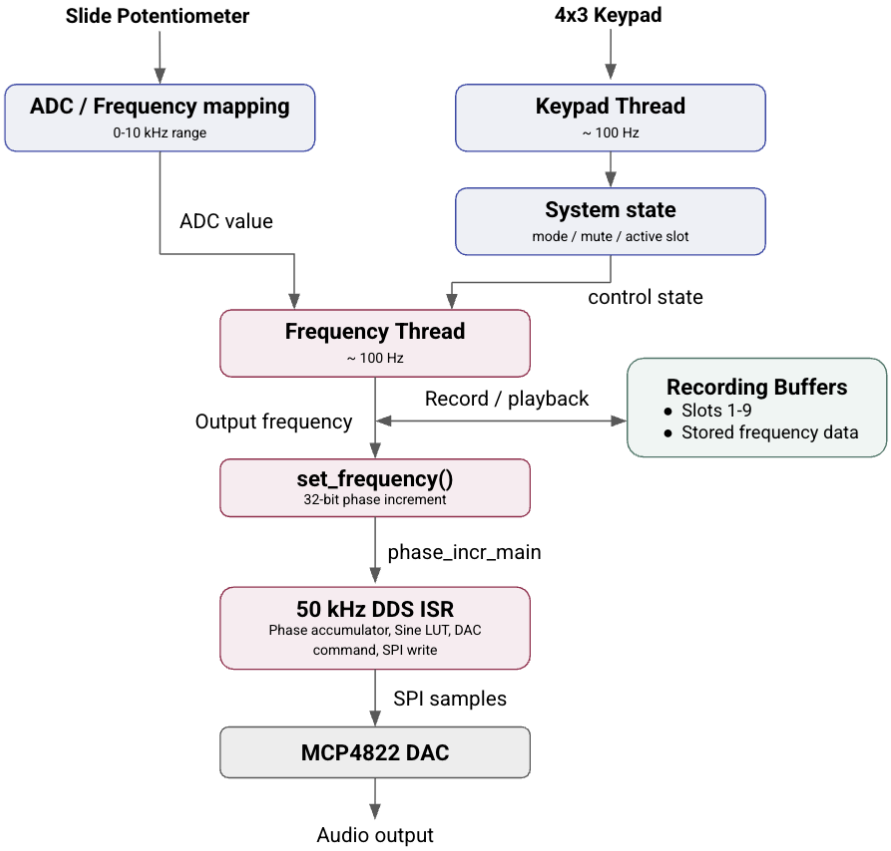


Figure 4: High-level software architecture.

2.3.1 DDS and Real-Time Audio

Audio generation was divided between a lower-rate control thread and a high-rate interrupt service routine (ISR). The control thread samples the slide potentiometer approximately every 10 ms (100 Hz) during live control and normal recording, using the RP2350's 12-bit ADC. The resulting value from 0–4095 is mapped linearly to a requested frequency from 0–10 kHz:

$$f_{\text{pot}} = \frac{\text{ADC}}{4095}(10,000 \text{ Hz}). \tag{1}$$

The requested frequency is converted into a phase increment for direct digital synthesis (DDS). DDS uses a 32-bit phase accumulator that advances by a fixed amount for every generated audio sample. For an output frequency f_{out} and synthesis rate f_s, the required phase increment is

$$\Delta\phi = \frac{f_{\text{out}}}{f_s} 2^{32}, \tag{2}$$

where the software uses the nominal rate $f_s = 50,000$ Hz. The upper eight bits of the accumulator are used to index a 256-entry sine lookup table. This avoids evaluating a sine function for every output sample while retaining sufficient resolution for tone generation.

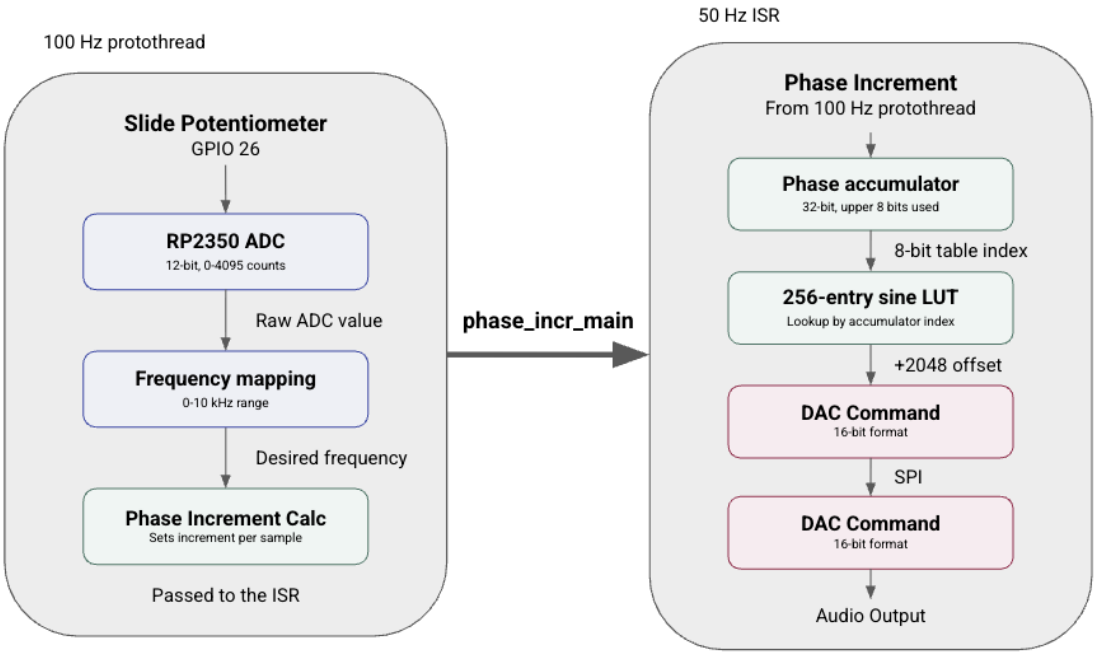


Figure 5: DDS signal-generation flow.

Waveform generation occurs inside a hardware timer ISR configured for a nominal 20 μs interval, corresponding to the 50 kHz synthesis rate. On each interrupt, the phase increment is added to the accumulator, the corresponding sine-table entry is retrieved, and the signed waveform is offset by 2048 to fit the 12-bit DAC range. The sample is then incorporated into the 16-bit MCP4822 command and transmitted over SPI to produce the analog audio output.

Separating these tasks allows slower operations such as ADC sampling, keypad processing, recording, and playback control to execute in protothreads without disrupting the fixed-rate audio generation. To evaluate the ISR duration relative to its 20 μs timing requirement, GPIO 2 was driven high upon ISR entry and low upon exit. The resulting pulse width was measured on an oscilloscope; the measured timing is discussed in the Results section.

2.3.2 Keypad Scanning and Debouncing

The keypad is read using a matrix scan, which lets 12 keys be sensed using only 7 GPIO pins (4 row pins and 3 column pins) instead of one pin per key. Each scan cycle drives one row pin low at a time while the others stay high, then checks the three column pins, which are configured as inputs with internal pull-up resistors, to see if any read low.

The row must be driven low rather than high because the column pins rely on these internal pull-ups, which hold them high by default. A press only produces a detectable change if the row being scanned is pulled low, since this allows a pressed key to pull its connected column down from high to low. If the row were also held high, a press would simply connect two already-high lines and produce no observable change. When key 1 is pressed, for example, it shorts row 1 to column 1, pulling column 1 low while row 1 is being driven low during that scan step.

Once a key is found, the row and column pattern is compared against a lookup table, `keycodes[]`, which maps each of the 12 raw 7-bit patterns to a logical key index: 0 corresponds to key 0, indices 1–9 correspond to keys 1–9, index 10 corresponds to `*`, and index 11 corresponds to `#`.

Mechanical switches do not transition cleanly between open and closed. A single physical press or release can cause the contact to chatter, briefly opening and closing several times before settling, a phenomenon known as bounce. If the firmware trusted every raw scan result, this could cause one physical press to be misread as several separate presses.

To prevent this, the system uses a four-state debounce machine. In `NOT_PRESSED`, the scan is checked repeatedly; when a valid key first appears, it is stored and the machine moves to `MAYBE_PRESSED`. If the next scan still shows the same key, the press is considered confirmed and the machine moves to `PRESSED`; if not, the reading was a bounce and the machine returns to `NOT_PRESSED`. Once in `PRESSED`, the machine stays there as long as the key keeps reading pressed. If the key stops reading as pressed, the machine moves to `MAYBE_NOT_PRESSED`, and confirms the release (returning to `NOT_PRESSED`) only if the next scan agrees; otherwise it returns to `PRESSED`, treating the change as bounce during release. A key must read the same result on two consecutive scans, spaced about 30 ms apart (based on the 30,000 µs yield in `protothread_keypad`), before a press or release is accepted.

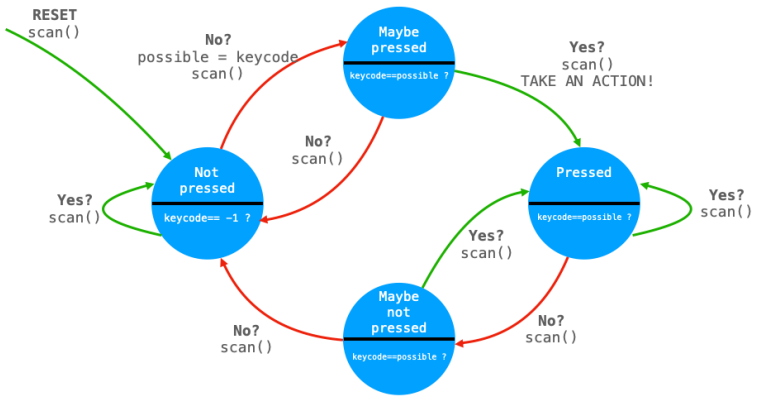


Figure 6: Keypad debounce state machine.

This distinction between a confirmed press and a confirmed release matters because different actions need different triggers. Mute toggling, arming record mode, and starting or stopping sequence collection are all triggered only on the confirmed press edge, so each fires exactly once per physical press rather than repeating on every scan while the key is held. Recording behaves differently: pressing * only arms record mode, and recording itself begins when a digit key (1–9) is subsequently pressed, selecting the slot to record into. Recording continues for as long as that key is held and ends only when the debounce state machine confirms release (`NOT_PRESSED`), since this duration cannot be captured by a single press-edge event the way the one-shot actions can.

The confirmed press edge of a digit key triggers different actions depending on context. Pressing a digit key immediately plays back the tone stored in that slot. If record mode has been armed via *, the same press instead selects the slot and begins recording. If sequence mode is active, the press appends the slot number to the sequence buffer without playing anything; playback of the full sequence only begins once # is pressed again to close the sequence.

2.3.3 Modes, Recording, and Playback

The synthesizer uses keypad-controlled states to manage live frequency control, recording, individual playback, and sequence composition. During normal operation, the frequency thread reads the potentiometer and maps its ADC value to a frequency between 0 and 10 kHz. Pressing `0` toggles the audio output independently of recording and playback. When muted, the software sets the DDS phase increment to zero, stopping waveform oscillation while allowing recording and playback processing to continue. Pressing * arms recording so that the next numbered key selects a recording slot. Recording does not automatically mute the output, allowing the user to hear the frequency changes being captured.

Recorded frequency trajectories are stored in `rec_buf[9][1000]`, with `rec_len[9]` tracking the number of valid samples in each slot. After recording is armed, a confirmed press of a key from 1 through 9 selects the corresponding slot, clears its previous length, and begins recording. While the key remains held, the frequency thread appends the current potentiometer frequency approximately every 10 ms during normal recording operation. A confirmed key release ends recording and clears the recording-arm state. Each slot can therefore store approximately 10 seconds of frequency changes. Once the buffer reaches 1,000 samples, additional samples are ignored. Recording into an occupied slot replaces its previous trajectory.

When neither recording nor sequence entry is armed, pressing a numbered key starts playback of that slot. The `play_sound` function selects the slot, resets the playback index, and sets the `playing` flag. The frequency thread then uses stored frequency values instead of the live potentiometer reading. During playback, the thread yields for 1 ms between updates, compared with 10 ms during normal recording. This produces approximately tenfold acceleration of the recorded trajectory, allowing relatively slow potentiometer movements to generate rapid birdsong-like sweeps. The stored frequency values remain unchanged; only the time between updates is shortened. When the slot finishes, output returns to live potentiometer control. An empty slot finishes without producing a recorded trajectory.

Sequence composition allows multiple recorded trajectories to be combined into a longer

pattern. Pressing `#` begins sequence entry and clears the previous sequence length. Subsequent numbered key presses append recording-slot numbers to `seq_buf` in the desired order, allowing the same slot to appear multiple times. Pressing `#` again ends sequence entry and starts playback. The frequency thread tracks the current sequence position using `seq_idx` and starts the next slot after the previous one finishes. Each slot uses the same accelerated playback timing as individual playback. After the final slot, sequence playback ends and live potentiometer control resumes. This approach allows reusable frequency gestures to be arranged into repeated call patterns without duplicating their recorded samples.

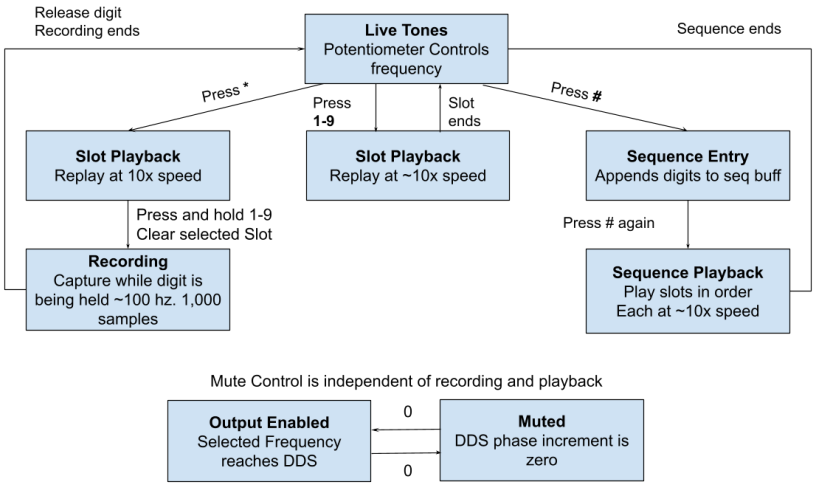


Figure 7: Operating mode transitions.

2.4 Testing, Debugging, and AI Use

2.4.1 Incremental Testing and Debugging

Following the course handout, we began by building the breadboard circuit connecting the RP2350 to the DAC over SPI, using the Week 1 demo code to generate a pure sine tone via direct digital synthesis. We confirmed correct operation by observing the generated waveform on the oscilloscope before connecting a speaker through an audio jack to confirm the tone was audible.

We then integrated the potentiometer using the ADC demo code, confirming that moving the pot produced a smooth, audible change in the output frequency, with no discontinuities or dropouts as the potentiometer was swept.

To characterize the timing of the synthesis ISR, we set a GPIO pin high on entry to the ISR and low on exit and measured the resulting pulse width on the oscilloscope, confirming the ISR completed well within the required sample period; see Section 4.1 and Figure 9 for the full measurement and analysis.

We integrated the matrix keypad following the course keypad reference page. Initial testing used the onboard LED as a simple indicator of whether key presses were being detected at all, before adding more detailed diagnostics. We then enabled USB serial output and ran a standalone keypad demo that printed the logical key index for each detected press. This surfaced a bug: pressing physical key 0 was being reported as logical index 2 rather than index 0. Because this diagnostic printed the resolved logical index directly, it narrowed the problem to the wiring, scanning, or key mapping before the mute-control logic. Tracing the connections revealed that the row and column pins were wired incorrectly, since two different keypad variants were distributed in lab and each expects a different pin mapping. After correcting the wiring to match our specific keypad, we reran the diagnostic and confirmed physical key 0 correctly reported index 0 and physical key * correctly reported index 10.

With the keypad correctly wired, we verified the remaining system behavior incrementally: mute/unmute toggling via key 0, arming recording with the * key and ending recording on a confirmed digit-key release, recording and playback of independent trajectories across multiple slots, overwriting a previously recorded slot with a new recording, and moving the potentiometer during playback to confirm that playback correctly ignores live pot input and continues playing the recorded trajectory unaffected.

For Week 3, we extended the Week 2 code by adding Compose Mode on top of the existing Tone/Record functionality. Initially, pressing the # key did not behave correctly. The bug was traced to the nested `if/else` inside the `k == POUND_KEY` branch, which is meant to distinguish a first press (start collecting a sequence) from a second press (stop collecting and play the sequence back). The nested conditional was structured incorrectly, so the toggle between collecting and playing a sequence did not behave as intended. Correcting the conditional structure resolved the issue; indentation was updated to make that structure clear. In C, indentation alone does not change control flow. After the correction, # correctly toggled between starting sequence collection and playing back the recorded sequence.

Table 1: Functional verification results.

Function	**Verification**
Live tone	Moved potentiometer and confirmed the output frequency changed smoothly
Mute	Pressed key 0 and confirmed the tone toggled on and off
Mode control	Pressed `*` to arm recording, held a numbered key to record, and confirmed that release ended recording
Recording and playback	Recorded independent frequency trajectories into multiple slots and confirmed each played back correctly
Overwrite	Re-recorded into an already-occupied slot and confirmed the old recording was replaced
Playback priority	Moved the potentiometer during active playback and confirmed playback correctly ignored live pot input and continued playing the recorded trajectory unaffected
Compose mode	Pressed `#` to collect a sequence of slots, then pressed `#` again and confirmed the sequence played back correctly

2.4.2 AI-Assisted Development and Verification

We used AI to review concepts relevant to the lab, including how the DDS phase accumulator advances through the sine lookup table and how the phase increment determines the output frequency. We also used AI to review how to wire components such as the potentiometer, including connecting its outer terminals to the supply and ground and its wiper to the ADC input. These explanations helped us understand the relationship between the hardware connections and the software that reads the potentiometer.

We also used AI for basic C syntax help, such as defining an `enum` and writing a `switch` statement. We checked our understanding by reviewing the code and testing the circuit, including observing how moving the potentiometer changed the generated tone.

3 Documentation

3.1 Hardware Reference

Table 2: RP2350 pin assignments.

GPIO	Function	Device
2	ISR timing	Oscilloscope
5	SPI CS	MCP4822
6	SPI SCK	MCP4822
7	SPI MOSI	MCP4822
9–12	Row outputs	Keypad
13–15	Column inputs	Keypad
25	Status LED	Pico 2
26	ADC0	Slide potentiometer

3.2 Software Organization

Table 3: Software task organization.

Task	Rate	Primary Function
Frequency thread	100 Hz / 1 kHz	Nominal live-control and recording / playback rates; ADC sampling and output selection
Keypad thread	100 Hz	Matrix scanning, debouncing, and user commands
DDS ISR	50 kHz	Phase accumulation, lookup, and DAC output

4 Results

4.1 Functional and Signal Results

The completed system successfully generated audio while supporting real-time frequency control, recording, and playback. In Tone Mode, moving the slide potentiometer produced a corresponding change in pitch across the programmed 0–10 kHz range. The keypad controls also operated as intended: the 0 key toggled the audio output, the $*$ key armed recording, and the numbered keys stored and replayed independent frequency trajectories. During playback, recorded frequency samples took priority over the live potentiometer input, with potentiometer control resuming after playback completed.

The output of the MCP4822 DAC was examined directly using an oscilloscope. Figure 8 shows a representative waveform captured while the system generated a swoop. The output followed the expected sinusoidal shape, while small discrete voltage steps were also visible in the waveform. These steps result from the DAC holding each digitally generated sample until the next sample arrives. The waveform therefore demonstrates the conversion of the

discrete DDS samples into an analog output. The changing frequency of the complete swoop is more clearly visualized using the spectrogram presented in Section 4.2.

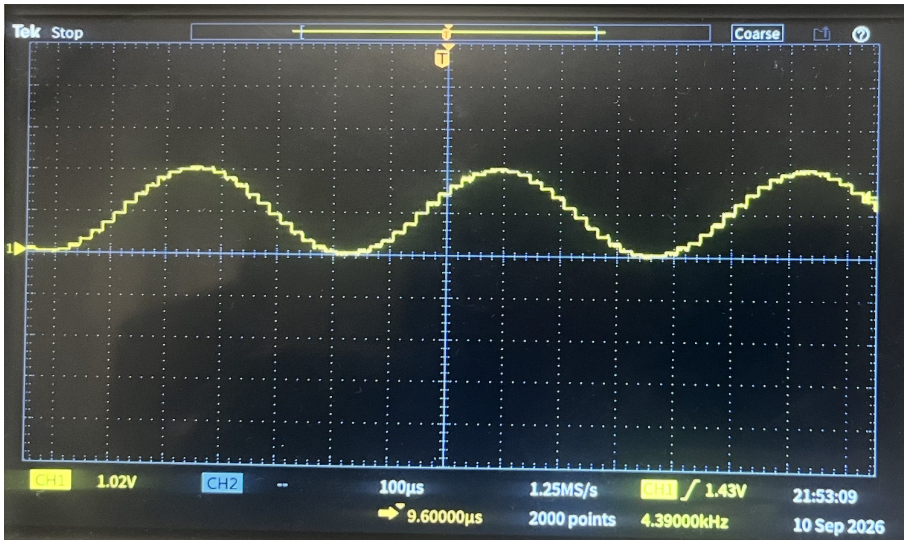


Figure 8: DAC output during a synthesized swoop.

Real-time performance was evaluated by instrumenting the DDS interrupt with GPIO 2. The pin was driven high immediately upon entering the ISR and low immediately before returning, so the pulse width measured the instrumented body of the interrupt. This excludes exception entry and return overhead and is not a worst-case execution-time bound. As shown in Figure 9, the measured ISR execution time was approximately 2.09 μs.

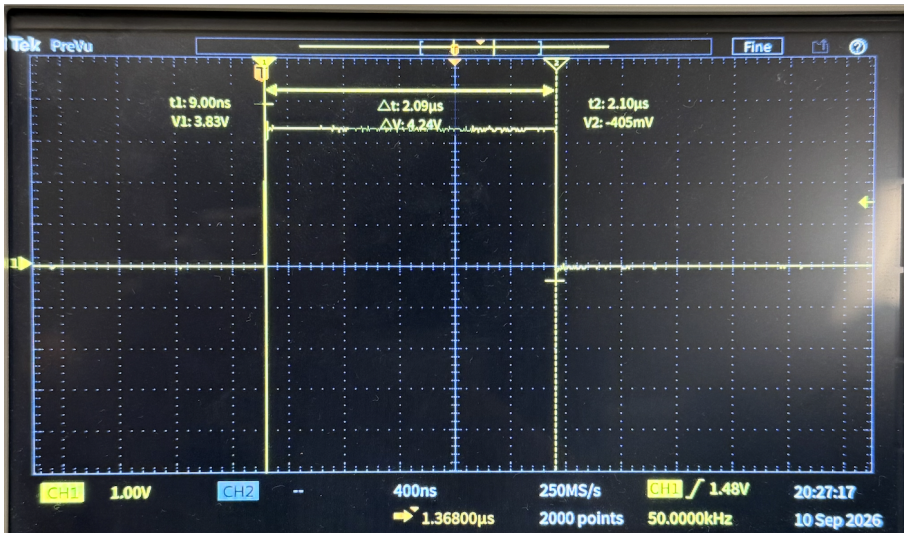


Figure 9: Measured ISR execution time.

A second oscilloscope measurement was used to verify the period between successive interrupts. The measured period was approximately 20.2 μs, as shown in Figure 10. This corresponds to a measured sampling frequency of

$$f_s = \frac{1}{20.2\ \mu\text{s}} \approx 49.5\ \text{kHz}, \tag{3}$$

which is within approximately 1% of the programmed 50 kHz synthesis rate. The 2.09 μs ISR therefore occupies approximately 10.3% of the measured interrupt period, leaving roughly 18.1 μs before the next interrupt. For the captured trace, the instrumented work completed within the measured interrupt period. Repeated measurements would be needed to characterize jitter or establish a worst-case timing bound.

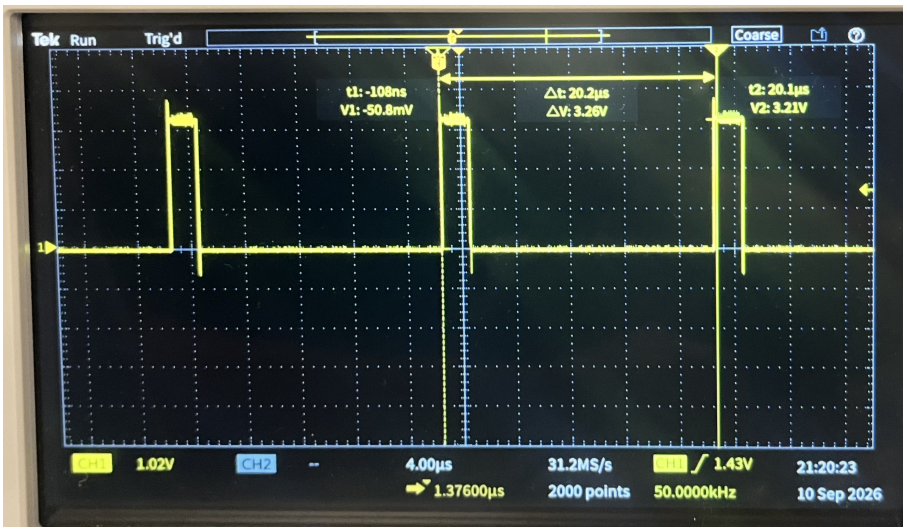


Figure 10: Measured DDS interrupt period.

Table 4 separates oscilloscope measurements from nominal parameters. Each slot holds 1000 frequency samples, corresponding to approximately 10 s of normal recording and approximately 1 s of accelerated playback. Sequence playback can concatenate or repeat slots. These durations are calculated estimates, not independently measured results.

Table 4: Timing measurements and nominal recording/playback parameters.

Quantity	**Value**	**Basis**
Instrumented ISR duration	2.09 μs	Oscilloscope trace
Interrupt period	20.2 μs	Oscilloscope trace
Effective sample rate	49.5 kHz	Reciprocal of measured period
Deviation from 50 kHz	-0.99%	Calculated from measured period
Instrumented duty fraction	10.3%	2.09/20.2
Live/record update interval	10 ms	Programmed yield
Playback update interval	1 ms	Programmed yield
Capacity per slot	1000 samples	Buffer limit

Because the DDS phase increment uses 50 kHz, a sustained sample rate of 49.5 kHz would produce frequencies approximately 0.99% below their requested values. This is an

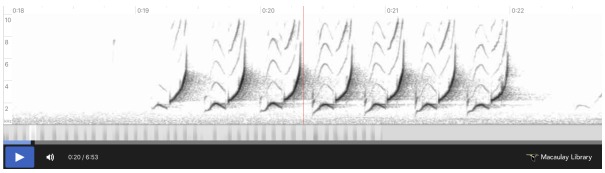
estimate based on the captured period, not a measured frequency-error sweep. The timer is rearmed relative to its current value inside the ISR, so interrupt latency and rearming time can contribute to the difference from the nominal period. The trace alone does not establish their individual contributions.

The functional tests establish observed operation under the tested conditions. Output-frequency error across the full range, timing jitter, and repeatability were not quantified in this report.

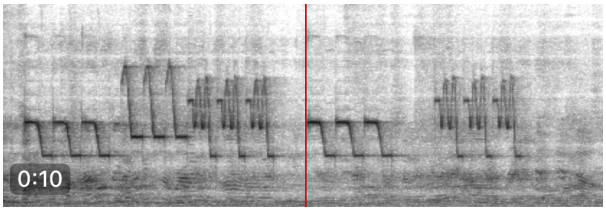
4.2 Birdsong and Spectrogram Evaluation

Figure 11 compares a reference cardinal call with the synthesized output. The reference sweeps across a wider frequency range, while the synthesized output covers a narrower, lower band and follows stepped segments. Manual potentiometer motion and discrete recording/playback updates can both contribute to the difference in sweep shape. Both signals show repeated calls; the synthesized repetition was produced by chaining recorded slots.

Faint parallel traces in the reference are consistent with harmonic content, whereas the synthesized output shows one dominant trace per tone, consistent with its sine-wave implementation. These observations suggest differences in frequency trajectory and spectral content, but do not isolate the cause of Merlin's recognition result.



(a) Reference cardinal call.



(b) Synthesized birdsong output.

Figure 11: Reference and synthesized spectrograms.

4.3 Merlin Evaluation and Limitations

Merlin Bird ID did not identify the synthesized sound as a northern cardinal or another species during our tests. As a control, a northern cardinal recording played from YouTube was identified after approximately 30 s in that test. The control demonstrates that recognition was possible in that instance; it does not establish a recognition rate for either signal.

Manually recorded gestures did not closely reproduce the reference call. Accelerated playback also made it difficult to anticipate the final note durations and sweep timing while moving the potentiometer. Differences in these patterns and harmonic content are plausible explanations for the lack of recognition, but the experiment did not separate their effects. Trial counts, sound levels, speaker-to-phone distance, and ambient conditions were not quantified in this report, so the result is a qualitative observation rather than a controlled classifier-performance measurement.

5 Conclusions

The system integrated real-time DDS audio generation, potentiometer control, keypad debouncing, nine recording slots, accelerated playback, and sequence composition. Functional testing verified live control, mute, recording, overwrite, playback priority, and sequence playback. The oscilloscope measurements showed an instrumented ISR duration of approximately 2.09 μs within a measured 20.2 μs interrupt period, supporting the feasibility of the audio-generation workload under the tested conditions.

The user could reuse recorded gestures and arrange them into longer patterns, but manually shaping a gesture for approximately tenfold accelerated playback made precise composition difficult. The synthesized calls were not recognized by Merlin, so recognizable cardinal imitation was not demonstrated. Recording capacity and discrete frequency updates also constrained individual gestures.

Future improvements would focus on matching the reference's sweep shapes and note timing, interpolating between stored frequency samples, and providing clearer feedback for recording and sequence entry. Bounds checking for sequence entry would improve robustness: the listed code appends entries without an explicit capacity check. Repeated timing and output-frequency measurements, followed by Merlin tests under recorded and consistent conditions, would make the evaluation more reproducible.

6 Work Distribution

- **Ariel Lai:** DDS generation, timer ISR and timing instrumentation, ADC/potentiometer integration, system bring-up and debugging, oscilloscope measurements, introduction, timing results, and final report integration.
- **Rumman Jan:** DAC and hardware wiring, keypad interface and scanning, debounce logic, hardware documentation, functional verification, and spectrogram analysis.
- **Ivan Zheng:** recording/playback buffers and control logic, operating-mode behavior, Compose Mode and playback timing, recording-capacity analysis, Merlin evaluation, and conclusion.
- **Collaborative:** Physical integration, birdsong tuning, debugging, end-to-end testing, and final report review.

A Complete Source Code

```
/*
 * Keypad-controlled DDS tone generator with record/playback
 *
 * Overview
 * --------
 * A 50 kHz timer ISR runs a direct digital synthesis (DDS) oscillator and pushes
 * samples to an SPI DAC. The audible frequency is controlled by a potentiometer
 * read at 100 Hz. Frequency "gestures" (pot sweeps) can be recorded into one of
 * nine slots and played back later, and slots can be chained into a sequence.
 *
```

```
 * Keypad map
 * ----------
 *   0         toggle the tone on/off (gate)
 *   1..9      context dependent:
 *               - after '*'  -> start recording into slot (key-1)
 *               - in seq mode -> append slot to the sequence
 *               - otherwise  -> play back slot (key-1)
 *   *         arm record mode (next digit selects the slot)
 *   #         first press: start collecting a sequence
 *             second press: stop collecting and play the sequence back
 *
 * Threads:
 *   protothread_freq    - 100 Hz: reads pot, records/plays back, sets DDS frequency
 *   protothread_keypad  - ~33 Hz: scans + debounces keypad, sets flags
 * ISR:
 *   alarm_irq           - 50 kHz: DDS sample out to DAC
 */
#include "hardware/gpio.h"
#include "hardware/timer.h"
#include "hardware/irq.h"
#include "hardware/adc.h"
#include "hardware/spi.h"
#include "hardware/clocks.h"

#include "pico/stdlib.h"

#include <stdio.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <stdbool.h>
#include "pt_cornell_rp2040_v1_4.h"

// ========================================
// === pins / hardware
// ========================================
#define LED_PIN 25
#define ADC_PIN 26    // analog input
#define ADC_MUX 0     // ADC input 0

#define ALARM_NUM 0                                                 // which hardware alarm drives the sample clock
#define ALARM_IRQ timer_hardware_alarm_get_irq_num(timer_hw, ALARM_NUM)

#define two32 4294967296.0   // 2^32, the full range of the phase accumulator
#define Fs 50000             // DDS sample rate (Hz), must match the alarm period
#define DELAY 20             // alarm reload in microseconds -> 1/20us = 50 kHz

// MCP4822-style DAC command words: config bits in the top nibble, 12-bit data below
#define DAC_config_chan_A 0b0011000000000000
#define DAC_config_chan_B 0b1011000000000000
#define DAC_CHANNEL DAC_config_chan_A

#define PIN_MISO 4
#define PIN_CS   5
#define PIN_SCK  6
#define PIN_MOSI 7
#define SPI_PORT spi0

#define ISR_GPIO 2   // toggled high for the duration of the ISR, for scoping ISR time

// Keypad occupies 7 consecutive GPIOs starting here:
//   BASE+0..BASE+3 are the four row drives (outputs)
//   BASE+4..BASE+6 are the three column senses (inputs, pulled up)
#define BASE_KEYPAD_PIN 9
#define KEYROWS         4
#define NUMKEYS         12

```

```
78  #define ASTERISK_KEY 10   // index into keycodes[] for '*'
79  #define POUND_KEY 11      // index into keycodes[] for '#'
80  // ========================================
81  // === recording storage
82  // ========================================
83  #define NUM_SLOTS   9      // one recording slot per digit key 1-9
84  #define MAX_SAMPLES 1000   // per slot; at 100 Hz this is 10 seconds
85  #define MAX_SEQUENCE 1000  // max slots chainable into one sequence
86
87  float rec_buf[NUM_SLOTS][MAX_SAMPLES];  // recorded frequency values, Hz
88  int   rec_len[NUM_SLOTS];               // how many samples each slot actually holds
89
90  int seq_buf[MAX_SEQUENCE];              // sequence of slot numbers (stored 1-based)
91
92  // ========================================
93  // === shared state (written by keypad thread, read by freq thread)
94  // ========================================
95  volatile bool toggle_frequency = true;   // tone gate (key 0)
96  volatile bool record_mode      = false;   // armed by '*'
97  volatile bool recording        = false;   // actively capturing pot values into rec_buf
98  volatile bool playing          = false;   // actively replaying one slot
99  volatile bool sequence_mode    = false;   // collecting slot numbers into seq_buf
100 volatile bool seq_playing      = false;   // stepping through seq_buf
101
102
103 volatile int  rec_slot         = 0;   // slot currently being recorded into
104 volatile int  play_slot        = 0;   // slot currently being played back
105 volatile int  play_idx         = 0;   // read cursor into rec_buf[play_slot]
106 volatile int seq_idx           = -1;  // position in seq_buf; -1 means "not started"
107 volatile int seq_len           = 0;   // number of entries collected in seq_buf
108
109 // ========================================
110 // === DDS
111 // ========================================
112 volatile unsigned int phase_accum_main;  // 32-bit phase, wraps naturally at 2^32
113 volatile unsigned int phase_incr_main;   // per-sample phase step; sets output frequency
114
115 // Convert a frequency in Hz into the phase step the ISR adds each sample.
116 void set_frequency(float frequency) {
117     phase_incr_main = (unsigned int)((frequency * two32) / Fs);
118 }
119
120 uint16_t DAC_data;   // one 16-bit SPI word: DAC config + sample
121
122 #define sine_table_size 256
123 volatile int sin_table[sine_table_size];   // one period of a sine, +/-2047
124
125 // ========================================
126 // === keypad tables
127 // ========================================
128 // keycodes[i] is the 7-bit (row drive | column sense) pattern seen when key i is
129 // down. Index order is 0,1,2,...,9,'*','#'.
130 unsigned int keycodes[NUMKEYS] = { 0x57, 0x6E, 0x5E, 0x3E, 0x6D,
131                                    0x5D, 0x3D, 0x6B, 0x5B, 0x3B,
132                                    0x67, 0x37 };
133 // One-low-at-a-time row drive patterns, walked during the scan.
134 unsigned int scancodes[KEYROWS] = { 0xE, 0xD, 0xB, 0x7 };
135 unsigned int button = 0x70;   // mask isolating the three column-sense bits
136
137 // ================================================
138 // === frequency thread (100 Hz)
139 // ================================================
140 // Single owner of set_frequency(). Reads pot, records or plays back,
141 // then writes exactly one frequency to the DDS.
142
143
144 // Kick off playback of a slot from the beginning.
145 void play_sound(int slot){
```

```
146     play_slot = slot;
147     playing = true;
148     play_idx = 0;
149 }
150 static PT_THREAD (protothread_freq(struct pt *pt))
151 {
152     PT_BEGIN(pt);
153 
154     // static so the values survive across PT_YIELD (protothreads have no stack)
155     static float pot_freq;
156     static float f;
157 
158     while(1) {
159         gpio_put(LED_PIN, !gpio_get(LED_PIN));   // heartbeat: shows the thread is running
160 
161         // Map the 12-bit ADC reading onto 0..10 kHz
162         pot_freq = ((float)adc_read() / 4095.0f) * 10000.0f;
163         f = pot_freq;   // default: play the live pot value
164 
165         if (recording) {
166             // Capture the live pot value; silently stop appending once the slot is full
167             if (rec_len[rec_slot] < MAX_SAMPLES) {
168                 rec_buf[rec_slot][rec_len[rec_slot]] = pot_freq;
169                 rec_len[rec_slot]++;
170             }
171         }
172         else if (seq_playing){
173             // Advance to the next slot in the sequence, but only once the
174             // previous slot has finished playing
175             if (!playing){
176                 seq_idx++;
177                 if (seq_idx<seq_len){
178                     play_slot = seq_buf[seq_idx]-1;   // seq_buf stores 1-based key numbers
179                     play_sound(play_slot);
180                 } else{
181                     // Ran off the end: reset and stop the sequence
182                     seq_idx=-1;
183                     seq_playing = false;
184                 }
185             }
186         }
187 
188         if (playing) {
189             // Override the pot with the next recorded sample
190             if (play_idx < rec_len[play_slot]) {
191                 f = rec_buf[play_slot][play_idx];
192                 play_idx++;
193             }
194             else {
195                 playing = false;   // slot exhausted; next pass falls back to the pot
196             }
197         }
198 
199         // Exactly one write to the DDS per pass. Gate closed -> DC (silence).
200         set_frequency(toggle_frequency ? f : 0.0f);
201 
202         if (playing){
203             PT_YIELD_usec(1000);   // 10 Hz
204         }
205         else{
206             PT_YIELD_usec(10000);   // 100 Hz
207         }
208 
209     }
210     PT_END(pt);
211 }
212 
213 // ================================================
```

```
214 // === debounce
215 // ================================================
216 // Four-state machine: a key must read the same on two consecutive scans
217 // (~30 ms apart) before a press or release is accepted.
218 typedef enum {NOT_PRESSED, MAYBE_PRESSED, PRESSED, MAYBE_NOT_PRESSED} ButtonState;
219
220 ButtonState currentState = NOT_PRESSED;
221 static int possible;   // candidate key index being confirmed
222
223 // Returns the key index on the press edge (MAYBE_PRESSED -> PRESSED), else -1.
224 int keypress_debounce(int i){
225     switch(currentState){
226         case NOT_PRESSED:
227             // First sighting of any key: remember it and wait for confirmation
228             if (i != -1){
229                 possible = i;
230                 currentState = MAYBE_PRESSED;
231             }
232             break;
233
234         case MAYBE_PRESSED:
235             // Same key twice in a row -> accept, and emit the edge exactly once
236             if (i == possible){
237                 currentState = PRESSED;
238                 return i;
239             }
240             else {
241                 currentState = NOT_PRESSED;   // glitch or bounce; discard
242             }
243             break;
244
245         case PRESSED:
246             // Held down. Anything else (including -1) starts the release check.
247             if (i != possible){
248                 currentState = MAYBE_NOT_PRESSED;
249             }
250             break;
251
252         case MAYBE_NOT_PRESSED:
253             if (i == possible){
254                 currentState = PRESSED;       // bounce during release; still held
255             }
256             else {
257                 currentState = NOT_PRESSED;   // release confirmed
258             }
259             break;
260     }
261     return -1;
262 }
263
264 // ================================================
265 // === keypad thread (~33 Hz)
266 // ================================================
267 static PT_THREAD (protothread_keypad(struct pt *pt))
268 {
269     PT_BEGIN(pt);
270
271     static int i;              // row index during scan, then key index
272     static int k;              // debounced key index on a press edge, else -1
273     static uint32_t keypad;    // raw 7-bit row|column reading
274
275     while(1) {
276
277         // Scan the keypad: drive one row low at a time and look for a column
278         // sense that has been pulled low by a closed switch.
279         for (i=0; i<KEYROWS; i++) {
280             gpio_put_masked((0xF << BASE_KEYPAD_PIN),
281                             (scancodes[i] << BASE_KEYPAD_PIN));
```

```
            sleep_us(1);   // let the line settle before sampling
            keypad = ((gpio_get_all() >> BASE_KEYPAD_PIN) & 0x7F);
            if ((~keypad) & button) break;   // some column is low -> key found in this row
        }
        if ((~keypad) & button) {
            // Look up the full row|column pattern in the key table
            for (i=0; i<NUMKEYS; i++) {
                if (keypad == keycodes[i]) break;
            }
            if (i==NUMKEYS) i = -1;   // unrecognised pattern (e.g. two keys down)
        }
        else i = -1;                  // nothing pressed

        // Debounce; k is the key index on a press edge, else -1
        k = keypress_debounce(i);

        if (k == ASTERISK_KEY) {
            // Arm recording; the next digit chooses the slot
            record_mode = true;
            printf("Record mode armed\n");
        }
        else if (k == POUND_KEY) {
            if (!sequence_mode) {
                sequence_mode = true;   // start collecting slot numbers
                seq_len = 0;
            } else {
                sequence_mode = false;  // done collecting -> play the sequence
                seq_idx = -1;
                seq_playing = true;
            }
        }
        else if (k == 0) {
            toggle_frequency = !toggle_frequency;   // mute / unmute
        }
        else if (k >= 1 && k <= 9) {
            if (record_mode) {
                // Select the slot, clear it, and start capturing
                rec_slot = k - 1;
                rec_len[rec_slot] = 0;
                recording = true;
            }
            else if(sequence_mode){
                seq_buf[seq_len++] = k;   // append to the sequence being built
            }
            else {
                play_sound(k-1);          // immediate playback of one slot
            }
        }

        // Recording runs for as long as the digit key is held; the confirmed
        // release (NOT_PRESSED) ends it.
        if (recording && currentState == NOT_PRESSED) {
            recording = false;
            record_mode = false;
            printf("Recorded %d samples\n", rec_len[rec_slot]);
        }

        PT_YIELD_usec(30000);   // ~33 Hz scan rate, also sets the debounce interval
    }
    PT_END(pt);
}

// ================================================
// === Interrupt Service Routine (50 kHz DDS)
// ================================================
static void alarm_irq(void) {
    gpio_put(ISR_GPIO, 1);   // raise timing pin: ISR entry

```

```
    hw_clear_bits(&timer_hw->intr, 1u << ALARM_NUM);          // acknowledge this alarm
    timer_hw->alarm[ALARM_NUM] = timer_hw->timerawl + DELAY;  // rearm for the next sample

    // DDS core: step the phase, use the top 8 bits as the table index
    phase_accum_main += phase_incr_main;
    int sine_value = sin_table[phase_accum_main >> 24];

    // Shift the signed sample into the DAC's unsigned 12-bit range and send it
    DAC_data = DAC_CHANNEL | ((sine_value + 2048) & 0xffff);
    spi_write16_blocking(SPI_PORT, &DAC_data, 1);

    gpio_put(ISR_GPIO, 0);   // drop timing pin: ISR exit
}

// ========================================
// === main
// ========================================
int main(){
    // Overclock FIRST, before any peripheral clocks are derived
    set_sys_clock_khz(150000, true);

    stdio_init_all();
    printf("\n\rPrototheads RP2040 v1.4\n\r");

    // ADC - potentiometer on ADC0
    adc_init();
    adc_gpio_init(ADC_PIN);
    adc_select_input(ADC_MUX);

    // LED
    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);
    gpio_put(LED_PIN, true);

    // SPI / DAC - 16-bit frames, mode 0, fast enough to finish well inside 20 us
    spi_init(SPI_PORT, 20000000);
    spi_set_format(SPI_PORT, 16, 0, 0, 0);
    gpio_set_function(PIN_MISO, GPIO_FUNC_SPI);
    gpio_set_function(PIN_SCK,  GPIO_FUNC_SPI);
    gpio_set_function(PIN_MOSI, GPIO_FUNC_SPI);
    gpio_set_function(PIN_CS,   GPIO_FUNC_SPI);

    // ISR timing pin
    gpio_init(ISR_GPIO);
    gpio_set_dir(ISR_GPIO, GPIO_OUT);
    gpio_put(ISR_GPIO, 0);

    // Sine table - one full period scaled to the DAC's half-range
    for (int ii = 0; ii < sine_table_size; ii++) {
        sin_table[ii] = (int)(2047 * sin((float)ii * 6.283 / (float)sine_table_size));
    }

    // Keypad GPIO: rows are driven outputs (idle high), columns are pulled-up inputs
    gpio_init_mask((0x7F << BASE_KEYPAD_PIN));
    gpio_set_dir((BASE_KEYPAD_PIN+4), GPIO_IN);
    gpio_set_dir((BASE_KEYPAD_PIN+5), GPIO_IN);
    gpio_set_dir((BASE_KEYPAD_PIN+6), GPIO_IN);
    gpio_set_dir_out_masked((0xF << BASE_KEYPAD_PIN));
    gpio_put_masked((0xF << BASE_KEYPAD_PIN), (0xF << BASE_KEYPAD_PIN));
    gpio_pull_up((BASE_KEYPAD_PIN+4));
    gpio_pull_up((BASE_KEYPAD_PIN+5));
    gpio_pull_up((BASE_KEYPAD_PIN+6));

    // DDS timer interrupt (arm last so nothing fires before setup is done)
    hw_set_bits(&timer_hw->inte, 1u << ALARM_NUM);
    irq_set_exclusive_handler(ALARM_IRQ, alarm_irq);
    irq_set_enabled(ALARM_IRQ, true);
    timer_hw->alarm[ALARM_NUM] = timer_hw->timerawl + DELAY;
```

```
418
419     // Threads
420     pt_add_thread(protothread_freq);
421     pt_add_thread(protothread_keypad);
422
423
424     pt_schedule_start;
425 }
```